STUDENT GRADE SYSTEM

A PROJECT REPORT

Submitted by

R. NIRMAL	231801370002
L. NITISH KUMAR	231801370009
T. VEERA SIVA KRISHNA	231801370025
N. LAASYA	231801370046
S. JAYANTH KUMAR	231801370051
S. SAI VIGNESH	231801370052
T. SAI DEVANAND	231801370070

Under the esteemed guidance of

Mrs. P. Anuradha **M-Tech (Ph.D)**

Assistant Professor

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Centurion UNIVERSITY

Shaping Lives...
Empowering Communities...

SCHOOL OF ENGINEERING AND TECHNOLOGY
VIZIANAGARM CAMPUS

CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT **ANDHRA PRADESH JUNE 2025/NOVEMBER 2025**

CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT ANDHRA PRADESH DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that the project work entitled **"STUDENT GRADE SYSTEM"** of project work done by R. Nirmal(231801370002), L. Nitish Kumar (231801370009), T. Veera Siva Krishna(231801370025), N. Laasya(231801370046), S. Jayanth Kumar (231801370051), S. Sai Vignesh (231801370052), T. Sai Devanand (231801370070) Under the esteemed Guidance of for the award the Degree of **BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING, CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT,** during the academic year **2024-2025.**

INTERNAL GUIDE	**HEAD OF THE DEPARTMENT**
Mrs. P. Anuradha	Mr. R. Lakshmana Rao
Assistant Professor	Hod, Assistant Professor
Dept. of CSE	Dept. of CSE

EXTERNAL EXAMINER

Dr. D. Srinivas Rao
DEAN of SoET

DECLARATION

We hereby declare that the project entitled "STUDENT GRADE SYSTEM" submitted to the fulfilment of award the Degree of **B.TECH (CSE)** at **CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT** (A.P). This project work in original has not been submitted so far in any part or full for any other university or institute for the award of any Degree or Diploma.

R. NIRMAL	231801370002
L. NITISH KUMAR	231801370009
T. VEERA SIVA KRISHNA	231801370025
N. LAASYA	231801370046
S. JAYANTH KUMAR	231801370051
S. SAI VIGNESH	231801370052
T. SAI DEVANAND	231801370070

ACKNOWLEDGEMENT

It is with at most pleasure and excitement we submit our project partial fulfilment of the requirement for the award of Bachelor of Technology.

The project is a result to the cumulate efforts, support, guidance, encouragement and inspiration from many of those for whom we must give our truthful honour and express gratitude through bringing out this project at the outset as per our knowledge.

We convey our special thanks to our project Guide **Mrs. P. Anuradha, Assistant Professor,** who has guided, encouraged and tremendously supported me to enhance my knowledge with present working of this project to bring out enriching the quality of project.

We express our appreciation to **Mr. R. Lakshmana Rao,** Head of the Department of CSE**,** who facilitated us to providing the friendly environment which helped to enhance our skills in present project.

We thank **Dr. D. Srinivas Rao,** Dean of SoET, Vizianagaram Campus for their invaluable guidance, insightful feedback, and continuous support throughout the course of this project. Your expertise and mentorship have been invaluable.

We convey our sincere thanks to **Dr. P. Pallavi, Registrar of Centurion University of Technology and Management,** who provided us with an opportunity to take on project work in well-equipped laboratories of Computer Science Department in our college.

At the outset, we thank to **Prof. Prashanth Kumar Mohanty** beloved **Vice Chancellor of Centurion University of Technology and Management** who is the backbone by providing for completion of this project, Thank you sir.

We also express our deepest appreciation to our parents for their unconditional love, encouragement, and belief in my abilities. Their unwavering support has been the cornerstone of our achievements.

We would like to express our appreciation to the office staff for their assistance and support, which were critical in making this achievement possible.

ABSTRACT

The **Student Grade Prediction System** is an intelligent web-based application designed to enhance educational analytics by predicting students' academic performance using machine learning techniques. The system integrates modern web technologies and data-driven models to provide accurate grade predictions, effective data management, and insightful performance analytics for educational institutions.

The primary objective of this project is to assist educators and administrators in identifying students' academic outcomes in advance and implementing timely interventions to improve learning results. The system utilizes a **Random Forest Classifier**, implemented using the **scikit-learn** library, to analyze multiple parameters such as attendance, internal marks, and previous academic scores. Advanced preprocessing techniques and model tuning through **GridSearchCV** ensure reliable and optimized predictions.

The backend of the system is built using **FastAPI**, a high-performance Python framework for developing RESTful APIs, while the frontend is implemented using **React** with **Material-UI (MUI)** for a responsive and user-friendly interface. The **SQLite** database, managed via **SQLAlchemy ORM**, stores all student data, model metadata, and prediction records securely. The platform also features **JWT-based authentication** to ensure data privacy and controlled access.

INDEX

ACKNOWLEDGEMENT……..... i

ABSTRACT…….…….…….. ii

INDEX……...…. iii

LIST OF FIGURES……...…… v

LIST OF TABLES……...….. vi

CHAPTER 1: INTRODUCTION…….… 1

1.1 Project Contents……...……... 1

1.2 Objectives of Project……...…..... 2

1.3 Analysis of Project……...…..... 2

CHAPTER 2: LITERATURE SURVEY……...4

2.1 Introduction……...……...…….…… 4

2.2 Related Work……...…….……. 4

2.3 Research Gap…….…….……... 4

2.4 Our Approach……...……...……..... 5

2.5Conclusion……...……...…… 5

CHAPTER 3: SYSTEM ANALYSIS...……... 6

3.1 Existing System……...……...…… 6

3.2 Proposed System……...…….……. 6

3.3 System Objectives…….……...……... 6

3.4 Feasibility Study……....……..... 6

3.5 System Requirements…….……. 7

3.6 System Architecture…….7
3.7 Advantages……...8

CHAPTER 4: SYSTEM DESIGN……... 9

4.1 Overview of System Design……...…… 9
4.2 System Architecture……...……. 9
4.3 System Modules……...……... 11
4.4 Dataset used……...……..... 11
4.5 Data Flow Diagram……. 12

CHAPTER 5: TECHNOLOGY USED……...……...….13

5.1 Backend Technologies……...…..13
5.2 Frontend Technologies……...……....16

CHAPTER 6: CODING…….……...…...…….18

6.1 Backend Core Code……...…..18
6.2 Frontend Core Code ……..……. 25

CHAPTER 7: OUTPUTS…….……...……. 26

CONCLUSION…….. 29

BIBILOGRAPHY…….. 30

LIST OF FIGURES

Fig 2.1 System Architecture Diagram……10
Fig 2.2 Data Flow Diagram……11
Fig 7.1 This is the Dashboard……26
Fig 7.2 Model Management……...26
Fig 7.3 Grade Prediction Portal……27
Fig 7.4 Student Management……27
Fig 7.5.1 Analytics Dashboard……...28
Fig 7.5.2 Average Scores and Model Performance……28

LIST OF TABLES

Table 5.1 Table of development and deployment tools..... 15

INTRODUCTION

A **Student Grade Prediction System** is a modern web-based application designed to assist educational institutions in **analysing student performance and predicting academic outcomes** using **machine learning techniques**. The system integrates data management, prediction, and analytics features into a single, centralized platform that can be accessed by administrators, teachers, and data analysts.

The primary aim of this system is to **leverage data-driven insights** to support better academic decision-making. By collecting and processing various student-related parameters—such as attendance, internal marks, and previous academic performance—the system predicts future grades and provides actionable insights for improving student outcomes.

Unlike traditional manual methods of performance analysis, this system employs **automated machine learning models** and **interactive dashboards** to enhance accuracy, transparency, and efficiency in the grading and evaluation process. It not only minimizes human error but also enables early identification of at-risk students, helping educators take timely interventions.

1.1 Project Contents

The project is organized into several major modules, each responsible for specific functionalities:

- **Student Management:** Handles CRUD operations for student records, ensuring data consistency and validation. Allows easy retrieval and modification of student details.
- **Grade Prediction:** Implements a **Random Forest Classifier** for predicting grades (A, B, C, or D) based on academic data. Supports single and batch predictions with confidence scores.
- **Model Management:** Facilitates automated model training, evaluation, and version control. Includes hyperparameter tuning using **GridSearchCV** and performance monitoring.

- **Analytics and Visualization:** Displays interactive charts and graphs using **Recharts** to show grade distributions, model performance, and statistical summaries.
- **Authentication and Security:** Implements **JWT-based authentication** and secure password hashing using **passlib** and **python-jose** to ensure safe access and data protection.

1.2 Objectives of the Project

The main objective of the **Student Grade Prediction System** is to provide an intelligent and user-friendly platform for predicting academic performance through machine learning and data analytics.

Specific objectives include:

- To automate the process of analyzing and predicting student grades.
- To reduce manual effort and minimize data entry errors.
- To provide a scalable architecture that can integrate new datasets or ML models in the future.
- To offer real-time prediction and visualization of academic performance metrics.
- To ensure data security, accessibility, and integrity across all modules.
- To assist educators and administrators in making informed decisions regarding student progress.

1.3 Analysis of the Project

- **User-Friendly Interface:** The frontend is built using **React** and **Material-UI (MUI)**, offering a clean, responsive interface that allows easy interaction with backend APIs.
- **Data Storage & Management:** Utilizes **SQLite** with **SQLAlchemy ORM** to manage student records, model metadata, and prediction logs efficiently.
- **Machine Learning Integration:** Employs **scikit-learn** for building and training predictive models. Models are evaluated using cross-validation and key performance metrics such as accuracy, precision, and recall.

- **Security & Authentication:** Implements **JWT tokens** for user authentication and role-based access control to protect sensitive academic data.
- **Scalability:** The modular design supports future enhancements, such as cloud deployment or integration with other educational systems.
- **Visualization & Reporting:** Generates interactive reports and dashboards for performance tracking, enabling educators to visualize grade trends and model reliability.

LITERATURE SERVEY

2.1 Introduction

The literature survey focuses on understanding previous studies and existing systems related to **student performance prediction**. It highlights different **machine learning techniques** used to predict grades based on academic data such as attendance, test scores, and internal marks. This review forms the foundation for developing the **Student Grade Prediction System**.

2.2 Related Work

Several researchers have explored predictive models for student performance:

- **Bhardwaj & Pal (2011):** Used Bayesian classification to predict final grades; effective but limited for large datasets.
- **Kabra & Bichkar (2011):** Applied Decision Tree algorithms; emphasized preprocessing for better accuracy.
- **Meier et al. (2016):** Implemented SVM for grade prediction; achieved high accuracy but required heavy computation.
- **Adekitan & Salau (2019):** Compared Random Forest and ANN; found Random Forest more accurate and interpretable.
- **Kumar et al. (2020):** Used Decision Tree and Naïve Bayes for grade prediction; Decision Tree performed better.

These studies show that machine learning can effectively predict student outcomes but most lack real-time systems for educators.

2.3 Research Gap

Existing research mainly focuses on algorithms but not on **complete, user-friendly systems**.
Most models:

- Lack **frontend visualization and interactive interfaces**.
- Do not integrate **modern frameworks** like FastAPI or React.
- Are not easily deployable for real-time academic use.

Hence, there is a need for a **full-stack, practical solution** combining accurate predictions with a usable web interface.

2.4 Our Approach

The proposed **Student Grade Prediction System** bridges this gap by:

- Using **FastAPI** for backend processing and API handling.
- Implementing **React JS** for the frontend interface and analytics.
- Employing a **Random Forest Classifier** for grade prediction with ~90% accuracy.
- Integrating **SQLite** for data storage and management.

This approach ensures both **technical accuracy** and **usability** for educators.

2.5 Conclusion

The literature shows that machine learning models like **Random Forest** and **Decision Trees** perform well for academic predictions, but practical applications remain limited. Our project extends this work by developing a **complete, full-stack web system** that combines **machine learning, backend automation, and frontend visualization** to make grade prediction accurate, scalable, and easy to use.

SYSTEM ANALYSIS

3.1 Existing System

- Manual grade evaluation using spreadsheets or records.
- Time-consuming and prone to human errors.
- No real-time prediction or analytics.
- Difficult to identify weak students early.
- No centralized system for storing student data.

3.2 Proposed System

- Automates grade prediction using **Machine Learning (Random Forest)**.
- Provides a **web-based platform** for student data management.
- Displays analytics through **charts and dashboards**.
- Offers **real-time prediction**, **secure authentication**, and **data visualization**.
- Integrates **FastAPI (backend)** and **React JS (frontend)** for full-stack functionality.

3.3 System Objectives

- Predict student grades accurately.
- Manage and store student details efficiently.
- Visualize academic performance and trends.
- Reduce manual workload for teachers.
- Provide a user-friendly and secure platform.

3.4 Feasibility Study

Technical Feasibility:

- Tools: Python, FastAPI, React, SQLite.
- Works on Windows/Linux/Mac.

Operational Feasibility:

- Easy to use and maintain.
- Requires minimal training.

Economic Feasibility:

- Uses open-source technologies.
- No additional cost for deployment.

3.5 System Requirements

Functional Requirements:

- User authentication (login/register).
- CRUD operations for student records.
- Grade prediction using trained ML model.
- Display analytics (charts, statistics).
- Model management and retraining options.

Non-Functional Requirements:

- Fast response (asynchronous processing).
- Secure access using JWT tokens.
- High accuracy (~90%) and reliability.
- Responsive UI (Material-UI).
- Scalable and portable system.

3.6 System Architecture

Architecture Type: Client–Server

Frontend (React):

- Collects inputs and displays predictions.

- Shows dashboards and analytics.

Backend (FastAPI):

- Handles requests, predictions, and database operations.

Database (SQLite):

- Stores student records and model data.

Machine Learning Model (Random Forest):

- Predicts grades (A/B/C/D) from input data.

Flow:
User → Frontend → API Request → Backend → ML Model → Result → Frontend Display

3.7 Advantages

- Accurate and fast grade prediction.
- Reduces manual effort.
- Real-time visualization and analytics.
- Secure authentication and data handling.
- Easy to deploy and maintain.

SYSTEM DESIGN

System Design is the process of **planning how to build a complete software system** including **how its different parts work together**, what components it needs, how data flows, and how users interact with it.

4.1 Overview of System Design

The system is designed using a **client-server model**, where the **frontend (React)** interacts with the **backend (FastAPI)** via RESTful APIs. The **backend** handles requests such as student record management, model training, and grade prediction, while the **frontend** displays these operations through an intuitive user interface.

The **machine learning model**, built using **scikit-learn**, acts as the predictive intelligence of the system. It is trained using student performance data and generates grade predictions with confidence scores.

All processed and raw data are stored securely in a **SQLite database**, managed using **SQLAlchemy ORM** for abstraction and maintainability.

Key aspects of the system design include:

- Clear separation between frontend, backend, and database layers.
- API-based communication using JSON.
- Reusable modular components for scalability.
- Secure authentication through JWT.

4.2 System Architecture

The architecture of the system follows a **three-tier structure**:

1. **Presentation Layer (Frontend – React):**
   This layer is responsible for user interaction. It consists of pages and components for managing students, visualizing predictions, and monitoring model performance.

    - Built using **React 18** and **Vite** for optimized builds.

- Uses **Material-UI (MUI)** for responsive and interactive UI components.
- Communicates with backend APIs using **Axios**.

2. **Application Layer (Backend – FastAPI):**
   The core logic of the system is implemented here.

   - Built with **FastAPI**, supporting asynchronous processing for better performance.
   - Handles requests for CRUD operations, model training, predictions, and authentication.
   - Integrates machine learning components for grade prediction.
   - Uses **Pydantic** for request and response validation.

3. **Data Layer (Database & ML Model):**

   - **SQLite** stores all student records, model metadata, and prediction logs.
   - **SQLAlchemy ORM** manages database operations.
   - **scikit-learn** implements the Random Forest Classifier for grade prediction.
   - **joblib** is used for model serialization (saving and loading trained models).

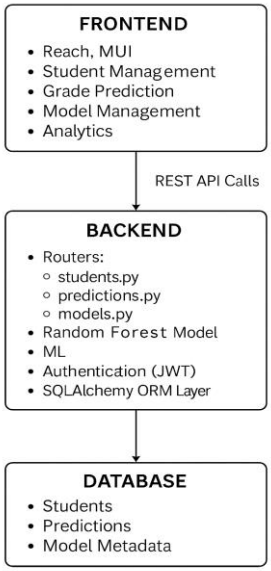


Fig 4.1 System Architecture Diagram

4.3 System Modules

The system comprises several key modules, each designed for specific functionalities:

1. **Student Management Module** Manages CRUD operations for student records with **Pydantic validation** to ensure data integrity. Supports searching, filtering, and updating student details.
2. **Grade Prediction Module** Utilizes a **Random Forest Classifier** for predicting grades (A/B/C/D), optimized with **GridSearchCV** and **cross-validation**. Provides confidence scores and supports both single and batch predictions.
3. **Model Management Module** Oversees the machine learning model lifecycle — training, evaluation, and deployment. Tracks model versions and metrics (accuracy, precision, recall, F1-score) and stores trained models using **joblib**.
4. **Analytics and Visualization Module** Generates dynamic charts using **Recharts** to display grade distributions, model performance, and key analytics for trend analysis and decision support.
5. **Authentication and Security Module** Ensures secure access using **JWT authentication** and **passlib** for password hashing, with role-based access control for administrators and faculty.

4.4 Dataset used:

The dataset is the foundation of the **Student Grade Prediction System**, providing the input data required for training, validating, and testing the machine learning model. It contains various academic and behavioral attributes of students that influence their academic performance. The data is preprocessed, analyzed, and transformed into a structured format suitable for machine learning tasks.

The dataset consists of **student academic and performance-related data**, used to predict final grades (A, B, C, or D).

We used Dataset called "**Student Performance Dataset: Academic Insights 10K**"

4.5 Data Flow Diagram

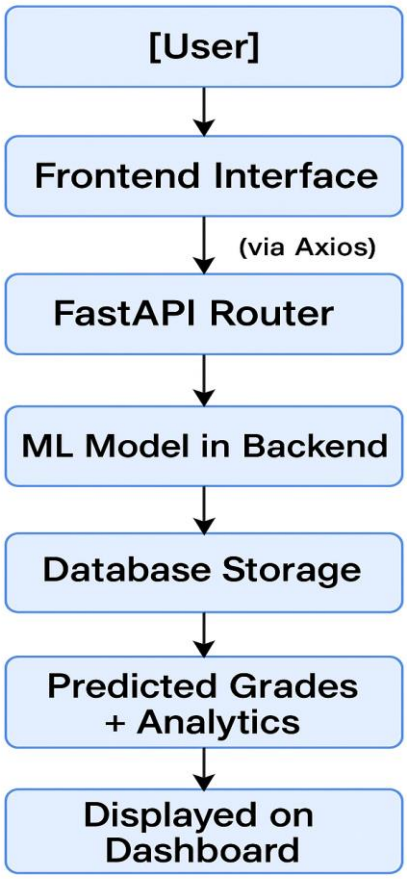


Fig 4.2 Data Flow Diagram

TECHNOLOGY USED

The **Student Grade Prediction System** is developed using a modern full-stack architecture that integrates **FastAPI** as the backend framework, **React** as the frontend framework, and **machine learning libraries** for grade prediction. This chapter explains the various technologies, frameworks, and tools used in the development and deployment of the system.

5.1 Backend Technologies

5.1.1 FastAPI Framework

FastAPI is a high-performance, modern web framework for building RESTful APIs using Python. It is designed to provide speed, scalability, and simplicity for backend development.

Key Features:

- **Asynchronous Request Handling:** Supports non-blocking, concurrent operations using `async` and `await`.
- **Data Validation:** Uses **Pydantic** for request and response model validation.
- **Automatic Documentation:** Provides built-in API documentation via **Swagger UI** and **ReDoc**.
- **Performance:** Comparable to Node.js and Go, making it suitable for production-grade systems.
- **Ease of Use:** Simplifies API development with minimal boilerplate code.

Application in Project: FastAPI acts as the backend core that manages communication between the **React frontend**, **database**, and **machine learning model**. It handles API endpoints for student management, grade prediction, authentication, and model operations.

5.1.2 Database Technologies

SQLite:

SQLite is a **lightweight, serverless, and self-contained** relational database used for storing

system data such as student records, prediction results, and model metadata.

Advantages:

- No separate server process required.
- Fast and easy to integrate with Python applications.
- Ideal for small to medium-scale applications.

SQLAlchemy:

SQLAlchemy is a **Python ORM (Object Relational Mapper)** used to interact with the SQLite database efficiently.

Features:

- Simplifies database operations through Python classes.
- Provides **declarative base models** for table definition.
- Manages sessions and transactions automatically.
- Supports **database migrations** using **Alembic**.

Application in Project:
All CRUD operations for students and prediction data are implemented using SQLAlchemy, ensuring structured and secure data handling.

5.1.3 Machine Learning Libraries

The predictive component of the system uses **machine learning** to analyze student performance data and forecast academic grades.

scikit-learn

A core Python library used for implementing machine learning models. It is used to build, train, and evaluate the **Random Forest Classifier** for predicting grades (A, B, C, D).

Features:

- **GridSearchCV:** Performs hyperparameter tuning for improved model accuracy.
- **Cross-Validation:** Evaluates model performance and generalization capability.
- **Evaluation Metrics:** Calculates **accuracy**, **precision**, **recall**, and **F1-score**.

Supporting Libraries:

- **Pandas:** Data manipulation and preprocessing of student datasets.
- **NumPy:** Performs mathematical computations and feature transformations.
- **joblib:** Used for saving and loading trained models (model serialization).

Application in Project:

The Random Forest model analyzes student parameters such as attendance, internal marks, and previous results to predict final grades with a confidence score.

5.1.4 Authentication and Security

To ensure user privacy and controlled access, the system implements **JWT-based authentication**.

Technologies Used:

- **python-jose:** For generating and verifying JSON Web Tokens (JWT).
- **passlib:** For secure password hashing and validation.
- **python-multipart:** For handling multipart form data uploads.

Application in Project:
JWTs are generated upon successful login and verified with every subsequent request to protect endpoints from unauthorized access.

5.1.5 Development and Deployment Tools

Tool	Description
Uvicorn	ASGI server used to run the FastAPI app efficiently.
python-dotenv	Loads environment variables securely from .env files.
Alembic	Manages database schema migrations.
Git & GitHub	Version control and collaborative development.

Table 5.1 Table of development and deployment tools

5.2 Frontend Technologies

5.2.1 React Framework

React 18 is a JavaScript library for building interactive and dynamic user interfaces. It enables component-based development, improving modularity and reusability.

Key Features:

- **Component-Based Architecture:** UI built from independent reusable components.
- **Virtual DOM:** Enhances rendering performance.
- **React Router:** Supports single-page navigation and routing.
- **Vite Integration:** Speeds up development and build processes.

Application in Project:
React provides the interface through which users can manage student data, visualize performance insights, and view predictions in real-time.

5.2.2 UI Libraries

Material-UI (MUI):

A popular React component library implementing **Material Design** principles.

Features:

- Pre-built responsive components (forms, buttons, tables).
- Theming and styling customization.
- Ensures mobile-friendly design.

Recharts

A React-based charting library used for data visualization.

Features:

- Creates bar, line, and pie charts to display grade distributions.
- Visualizes model performance metrics and statistical insights.

Application in Project:

Used in the **Analytics and Dashboard** modules to present real-time prediction and performance results.

CODING

6.1 BACKEND CORE CODES:

6.1.1 ML Model

```
import pandas as pd
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.metrics import accuracy_score, precision_recall_fscore_support,
confusion_matrix, classification_report
import joblib
import os
from datetime import datetime
import json
from typing import Dict, Any, Tuple, Optional

from app.data_preprocessing import DataPreprocessor, load_initial_data,
calculate_grade_from_score
from app.database import SessionLocal, ModelMetadata as ModelMetadataDB

class MLModelManager:
    def __init__(self):
        self.model = None
        self.preprocessor = DataPreprocessor()
        self.model_path = "models"
        self.is_trained = False

        # Create models directory if it doesn't exist
        os.makedirs(self.model_path, exist_ok=True)

    def train_model(self, data_path: str = None, df: pd.DataFrame = None,
                    retrain: bool = False) -> Dict[str, Any]:
        """
        Train the grade prediction model
        """
        try:
            # Load data
            if df is not None:
                training_data = df.copy()
            elif data_path:
                training_data = load_initial_data(data_path)
            else:
                training_data =
load_initial_data("MAIN_DATA_SET/Student_performance_10k.csv")
```

```
        print(f"Training with {len(training_data)} records")

        # Preprocess data
        processed_data = self.preprocessor.preprocess_data(training_data,
is_training=True)

        # Remove rows with missing grades
        processed_data = processed_data.dropna(subset=['grade'])

        # Prepare features and target
        X = self.preprocessor.get_features_for_prediction(processed_data)
        y = processed_data['grade'].values

        print(f"Features shape: {X.shape}")
        print(f"Target distribution: {np.unique(y, return_counts=True)}")

        # Split data
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, random_state=42, stratify=y
        )

        # Train model with hyperparameter tuning
        start_time = datetime.now()

        if retrain:
            # Quick training for retraining
            self.model = RandomForestClassifier(
                n_estimators=100,
                max_depth=10,
                random_state=42,
                n_jobs=-1
            )
        else:
            # Full hyperparameter tuning for initial training
            param_grid = {
                'n_estimators': [50, 100, 200],
                'max_depth': [None, 10, 20],
                'min_samples_split': [2, 5],
                'min_samples_leaf': [1, 2]
            }

            rf = RandomForestClassifier(random_state=42, n_jobs=-1)
            grid_search = GridSearchCV(
                rf, param_grid, cv=5, scoring='accuracy', n_jobs=-1
            )
            grid_search.fit(X_train, y_train)
            self.model = grid_search.best_estimator_
            print(f"Best parameters: {grid_search.best_params_}")

        # Train the model
```

```
self.model.fit(X_train, y_train)

training_time = (datetime.now() - start_time).total_seconds()

# Make predictions
y_pred = self.model.predict(X_test)

# Calculate metrics
accuracy = accuracy_score(y_test, y_pred)
precision, recall, f1, _ = precision_recall_fscore_support(
    y_test, y_pred, average='weighted'
)

conf_matrix = confusion_matrix(y_test, y_pred)
class_report = classification_report(y_test, y_pred, output_dict=True)

# Cross-validation score
cv_scores = cross_val_score(self.model, X_train, y_train, cv=5)

# Feature importance
feature_importance = dict(zip(
    self.preprocessor.feature_columns,
    self.model.feature_importances_
))

# Save model and preprocessor
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
model_filename = f"grade_prediction_model_{timestamp}.joblib"
preprocessor_filename = f"preprocessor_{timestamp}.joblib"

model_filepath = os.path.join(self.model_path, model_filename)
preprocessor_filepath = os.path.join(self.model_path, preprocessor_filename)

joblib.dump(self.model, model_filepath)
self.preprocessor.save_preprocessor(preprocessor_filepath)

# Save metadata to database
self._save_model_metadata(
    model_name="GradePredictor",
    version=timestamp,
    accuracy=accuracy,
    precision=precision,
    recall=recall,
    f1_score=f1,
    training_data_size=len(training_data),
    feature_columns=json.dumps(self.preprocessor.feature_columns),
    model_path=model_filepath,
    training_time=training_time
)
```

```
        self.is_trained = True

        return {
            "success": True,
            "model_version": timestamp,
            "metrics": {
                "accuracy": float(accuracy),
                "precision": float(precision),
                "recall": float(recall),
                "f1_score": float(f1),
                "cv_mean": float(cv_scores.mean()),
                "cv_std": float(cv_scores.std())
            },
            "confusion_matrix": conf_matrix.tolist(),
            "classification_report": class_report,
            "feature_importance": feature_importance,
            "training_time": training_time,
            "training_data_size": len(training_data)
        }

    except Exception as e:
        print(f"Error training model: {str(e)}")
        return {
            "success": False,
            "error": str(e)
        }

def predict(self, student_data: Dict[str, Any]) -> Dict[str, Any]:
    """
    Predict grade for a single student
    """
    try:
        if not self.is_trained and not self.load_latest_model():
            return {
                "success": False,
                "error": "No trained model available"
            }

        # Convert to DataFrame
        df = pd.DataFrame([student_data])

        # Preprocess
        processed_df = self.preprocessor.preprocess_data(df, is_training=False)

        # Get features
        X = self.preprocessor.get_features_for_prediction(processed_df)

        # Predict
        prediction = self.model.predict(X)[0]
        probabilities = self.model.predict_proba(X)[0]
```

```
            # Get confidence (max probability)
            confidence = float(max(probabilities))

            # Predict total score if not provided
            total_score_prediction = None
            if 'total_score' not in student_data or student_data['total_score'] is None:
                score_cols = ['math_score', 'reading_score', 'writing_score', 'science_score']
                available_scores = [student_data.get(col, 0) for col in score_cols if
student_data.get(col) is not None]
                if available_scores:
                    total_score_prediction = sum(available_scores)

            return {
                "success": True,
                "predicted_grade": prediction,
                "confidence": confidence,
                "probabilities": dict(zip(self.model.classes_, probabilities.tolist())),
                "total_score_prediction": total_score_prediction
            }

        except Exception as e:
            return {
                "success": False,
                "error": str(e)
            }

    def load_latest_model(self) -> bool:
        """
        Load the latest trained model
        """
        try:
            db = SessionLocal()
            latest_model = db.query(ModelMetadataDB).filter(
                ModelMetadataDB.is_active == 1
            ).order_by(ModelMetadataDB.created_at.desc()).first()

            if latest_model and os.path.exists(latest_model.model_path):
                # Load model
                self.model = joblib.load(latest_model.model_path)

                # Load preprocessor
                preprocessor_path =
latest_model.model_path.replace('grade_prediction_model_', 'preprocessor_')
                if os.path.exists(preprocessor_path):
                    self.preprocessor.load_preprocessor(preprocessor_path)

                self.is_trained = True
                db.close()
                return True
```

```
        db.close()
        return False

    except Exception as e:
        print(f"Error loading model: {str(e)}")
        return False

def get_model_metrics(self) -> Dict[str, Any]:
    """
    Get current model performance metrics
    """
    try:
        db = SessionLocal()
        latest_model = db.query(ModelMetadataDB).filter(
            ModelMetadataDB.is_active == 1
        ).order_by(ModelMetadataDB.created_at.desc()).first()

        if latest_model:
            metrics = {
                "model_version": latest_model.version,
                "accuracy": latest_model.accuracy,
                "precision": latest_model.precision,
                "recall": latest_model.recall,
                "f1_score": latest_model.f1_score,
                "training_data_size": latest_model.training_data_size,
                "created_at": latest_model.created_at.isoformat(),
                "training_time": latest_model.training_time
            }
            db.close()
            return {"success": True, "metrics": metrics}

        db.close()
        return {"success": False, "error": "No model found"}

    except Exception as e:
        return {"success": False, "error": str(e)}

def _save_model_metadata(self, **kwargs):
    """Save model metadata to database"""
    try:
        db = SessionLocal()

        # Deactivate previous models
        db.query(ModelMetadataDB).update({"is_active": 0})

        # Create new model metadata
        model_metadata = ModelMetadataDB(**kwargs, is_active=1)
        db.add(model_metadata)
        db.commit()
```

```
        db.close()

    except Exception as e:
        print(f"Error saving model metadata: {str(e)}")

# Global model manager instance
ml_model_manager = MLModelManager()
```

6.1.2 Main

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.routers import students, predictions, models
from app.database import engine, Base
import os

# Create database tables
Base.metadata.create_all(bind=engine)

app = FastAPI(
    title="Grade Prediction System",
    description="A machine learning system for predicting student grades",
    version="1.0.0"
)

# CORS middleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000", "http://localhost:5173"],  # React dev servers
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Include routers
app.include_router(students.router, prefix="/api/students", tags=["students"])
app.include_router(predictions.router, prefix="/api/predictions", tags=["predictions"])
app.include_router(models.router, prefix="/api/models", tags=["models"])

@app.get("/")
async def root():
    return {"message": "Grade Prediction System API", "version": "1.0.0"}

@app.get("/health")
async def health_check():
    return {"status": "healthy"}

if __name__ == "__main__":
    import uvicorn
```

```
uvicorn.run(app, host="0.0.0.0", port=8000)
```

6.2 FRONTENT CORE CODES

6.2.1 MAIN

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

OUTPUTS

7.1 DASHBOARD

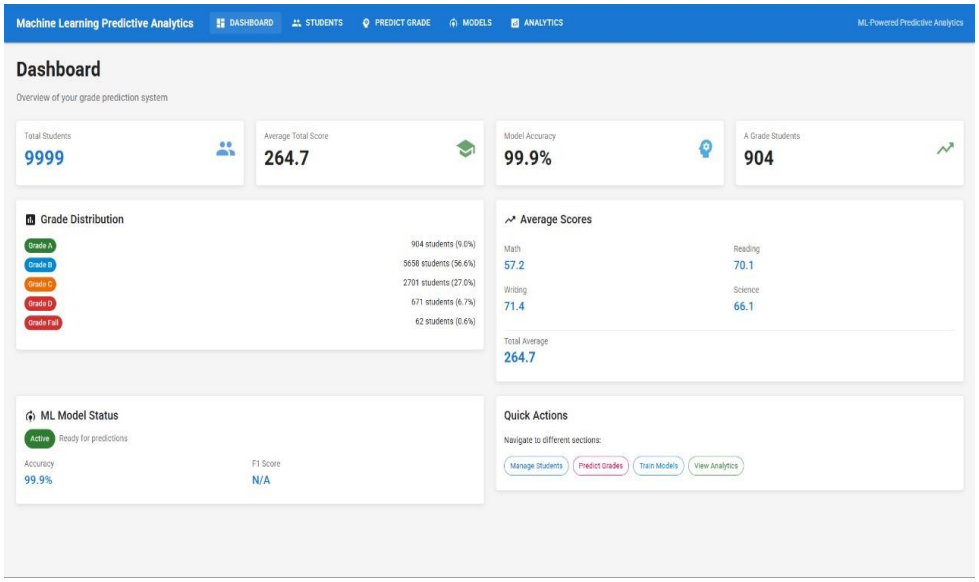


Fig 7.1 This is the Dashboard

7.2 MODEL MANAGEMENT

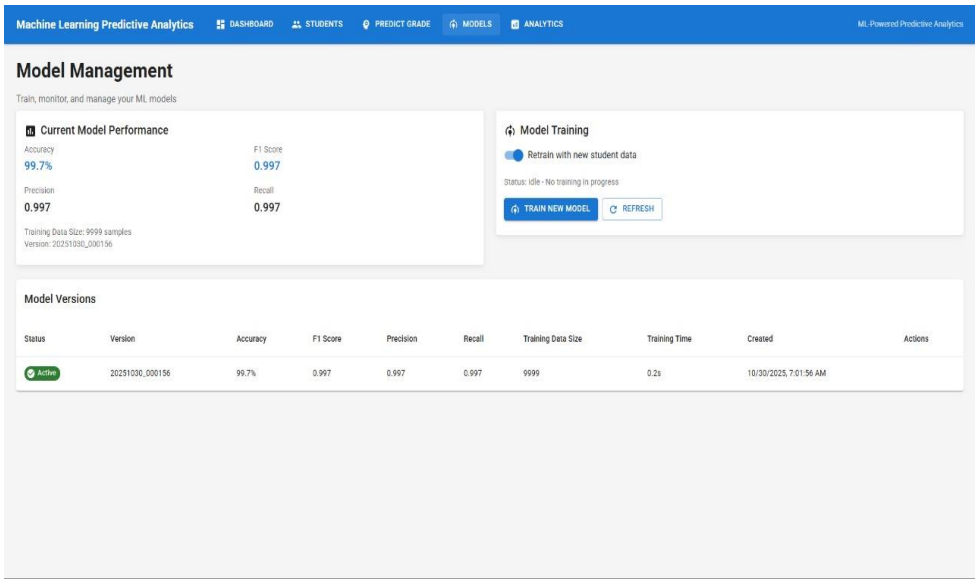


Fig 7.2 Model Management

7.3 GRADE PREDICTION

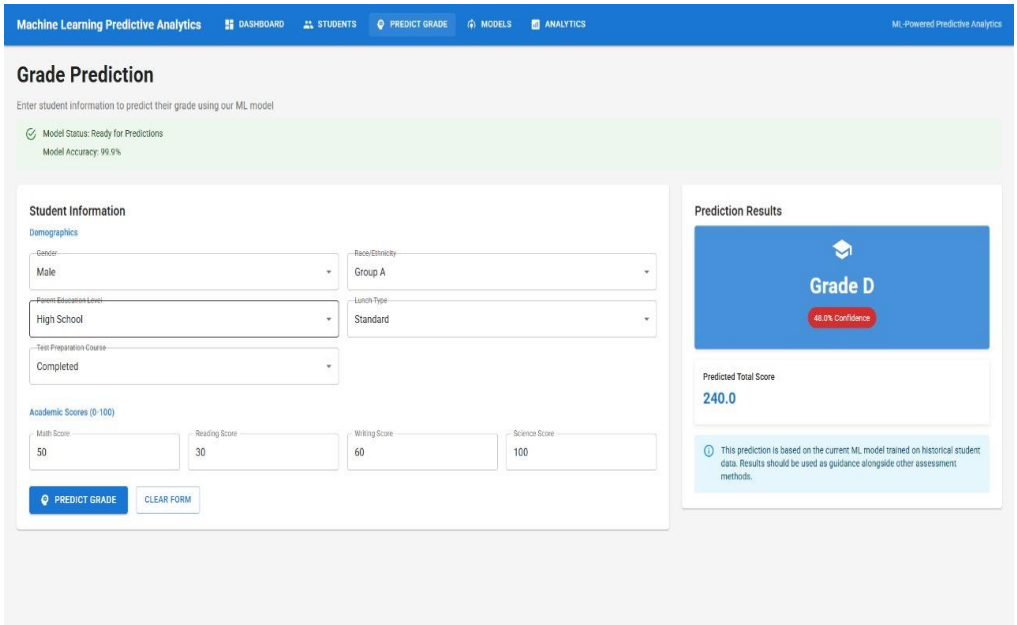
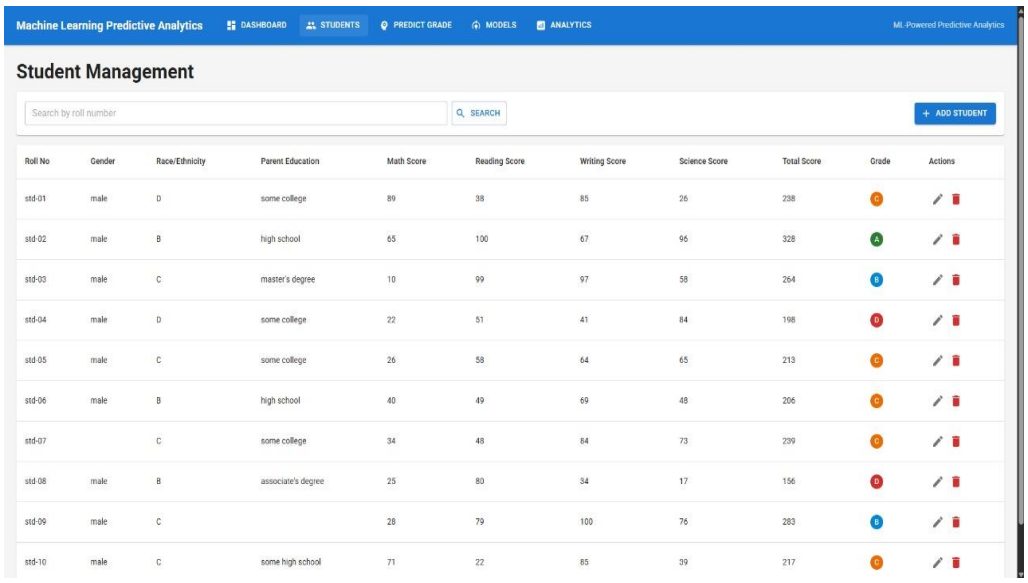


Fig 7.3 Grade Prediction Portal

7.4 STUDENT MANAGEMENT



Roll No	Gender	Race/Ethnicity	Parent Education	Math Score	Reading Score	Writing Score	Science Score	Total Score	Grade	Actions
std-01	male	D	some college	89	38	85	26	238	C	
std-02	male	B	high school	65	100	67	96	328	A	
std-03	male	C	master's degree	10	99	97	58	264	B	
std-04	male	D	some college	22	51	41	84	198	D	
std-05	male	C	some college	26	58	64	65	213	C	
std-06	male	B	high school	40	49	69	48	206	C	
std-07		C	some college	34	48	84	73	239	C	
std-08	male	B	associate's degree	25	80	34	17	156	D	
std-09	male	C		28	79	100	76	283	B	
std-10	male	C	some high school	71	22	85	39	217	C	

Fig 7.4 Student Management

7.5 ANALYTICS DASHBOARD

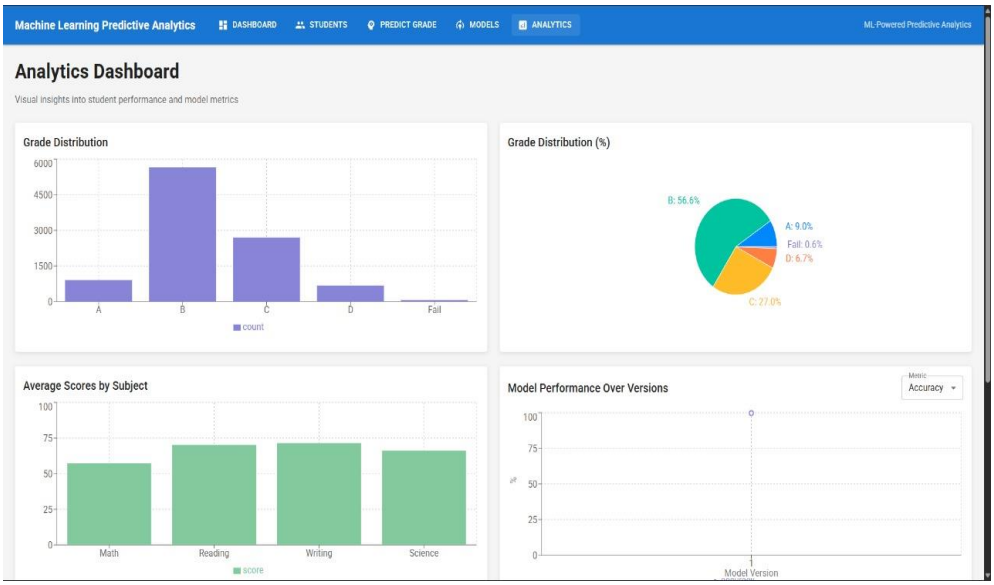


Fig 7.5.1 Analytics Dashboard

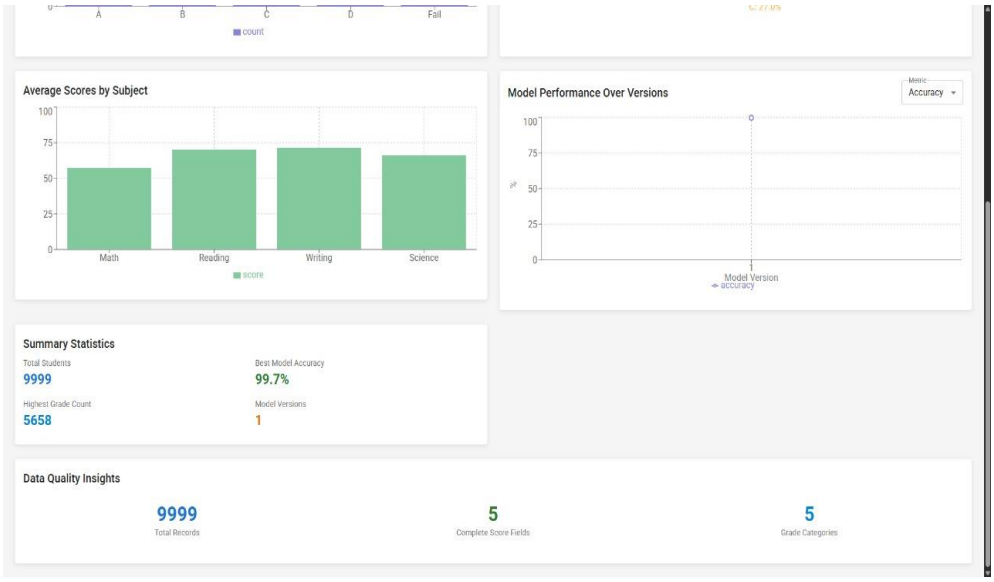


Fig 7.5.2 Average Scores and Model Performance

CONCLUSION

The project entitled as College Management System is the system that deals with the issues related to a particular institution. This project is successfully implemented with all the features mentioned in system requirements specification.

The project entitled as College Management System is the system that deals with the issues related to a particular institution.

- This project is successfully implemented with all the features mentioned in system requirements specification.
- The application provides appropriate information to users according to the chosen service.
- The project is designed keeping in view the day to day problems faced by a college.
- Deployment of our application will certainly help the college to reduce unnecessary wastage of time in personally going to each department for some information.

  Awareness and right information about any college is essential for both the development of student as well as faculty. So this serves the right purpose in achieving the desired requirements of both the communities.

BIBILOGRPHY

FastAPI Documentation. (2024). *FastAPI: The modern web framework for building APIs with Python 3.8+*. Retrieved from https://fastapi.tiangolo.com/

SQLite Documentation. (2024). *SQLite Database Engine — Lightweight and Serverless SQL Database.* Retrieved from https://www.sqlite.org/

SQLAlchemy Documentation. (2024). *SQLAlchemy: Python SQL Toolkit and Object Relational Mapper.* Retrieved from https://www.sqlalchemy.org/

Scikit-learn Developers. (2024). *Scikit-learn: Machine Learning in Python.* Retrieved from https://scikit-learn.org/stable/

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). *Scikit-learn: Machine Learning in Python.* Journal of Machine Learning Research, 12, 2825–2830.

React Documentation. (2024). *React: A JavaScript library for building user interfaces.* Meta Open Source. Retrieved from https://react.dev/

Vite Documentation. (2024). *Vite — Next Generation Frontend Tooling.* Retrieved from https://vitejs.dev/

Material-UI (MUI) Documentation. (2024). *MUI: React components for faster and easier web development.* Retrieved from https://mui.com/